CXL Emulation using QEMU on Windows Subsystem for Linux2 (WSL2)

1. Introduction

CXL (Compute Express Link) is an open standard for high-speed, cache-coherent CPU to device and CPU to memory interconnects, built on top of the PCIe physical layer. Since real CXL hardware is not yet widely available and is very expensive, software emulation using QEMU is the standard approach for development and testing.

This document describes the complete process of setting up a CXL emulation environment on a Windows laptop using WSL2 (Windows Subsystem for Linux 2), including all steps taken, the final architecture achieved, what worked, and known limitations.

2. Environment and Architecture

2.1 Overall Setup

Since real CXL hardware requires Linux and KVM, and the host machine runs Windows, the following layered architecture was used:

```
Windows 11 Host Laptop
    └── WSL2 Engine (Ubuntu 24.04 LTS)              Build environment
            └── QEMU 10.2.91 (built from source)    CXL hardware emulator
                    └── Ubuntu 24.04 Guest VM        CXL software stack
                            └── CXL Type 3 Device   512MB emulated CXL memory
```

2.2 CXL Topology Emulated

Component	QEMU Device	Description
CXL Host Bridge	`pxb-cxl`	Gateway between PCIe and CXL world (bus_nr=12)
CXL Root Port	`cxl-rp`	PCIe style slot for one CXL device (port=0, slot=2)
CXL Type 3 Device	`cxl-type3`	512MB volatile CXL memory expansion device
Memory Backend	`cxl-mem1.img`	512MB file on disk backing the CXL device memory
Label Storage Area	`cxl-label1.img`	4KB metadata store for the CXL device
Fixed Memory Window	`cxl-fmw`	4GB address space reserved for CXL memory

3. Complete Setup Steps

Step 1: WSL2 Setup on Windows

WSL2 was already installed on the Windows host. Ubuntu 24.04 LTS was installed as the Linux distribution.

```
wsl --install -d Ubuntu-22.04
wsl --list --verbose
```

Step 2: Install Build Dependencies

All required development libraries and tools were installed inside WSL2 Ubuntu:

```
sudo apt update && sudo apt upgrade -y
sudo apt install -y build-essential git ninja-build python3 python3-pip \
  libglib2.0-dev libfdt-dev libpixman-1-dev zlib1g-dev \
  libpmem-dev libdaxctl-dev libnuma-dev \
  flex bison libelf-dev libssl-dev bc \
  pkg-config libslirp-dev qemu-utils ndctl
```

Step 3: Why QEMU Was Built from Source

QEMU can be installed directly from the Ubuntu package manager using apt install qemu. However, this approach was not suitable for CXL emulation. CXL emulation in QEMU is a rapidly evolving feature. The pre-packaged version available via apt is always several versions behind and does not include the required CXL device properties. Building from the latest source code was the only reliable path to working CXL emulation.

The build commands used:

```
mkdir ~/cxl && cd ~/cxl
git clone https://gitlab.com/qemu-project/qemu.git
cd qemu && git submodule update --init --recursive
mkdir build && cd build
../configure --target-list=x86_64-softmmu --enable-libpmem --enable-slirp
make -j$(nproc)
```

Result: QEMU 10.2.91 built successfully with libpmem and libdaxctl support confirmed.

Step 4: Build CXL-Enabled Linux Kernel

The mainline Linux kernel was cloned and configured with all necessary CXL drivers:

```
cd ~/cxl
git clone https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git
cd linux && make defconfig
```

The following CXL specific kernel configuration options were added:

```
CONFIG_CXL_BUS=y          # Core CXL bus driver
CONFIG_CXL_MEM=y          # CXL memory device driver
```

```
CONFIG_CXL_PORT=y         # CXL port driver
CONFIG_CXL_ACPI=y         # CXL ACPI discovery
CONFIG_CXL_PMEM=y         # CXL persistent memory
CONFIG_ACPI_HMAT=y        # ACPI HMAT support
CONFIG_MEMORY_HOTPLUG=y   # Memory hotplug
CONFIG_ZONE_DEVICE=y      # Zone device support
CONFIG_DEV_DAX=y          # DAX device support
CONFIG_DEV_DAX_HMEM=y     # HMEM DAX support
CONFIG_NUMA=y             # NUMA topology
```

```
make olddefconfig
make -j$(nproc) bzImage
```

Result: Linux 7.0.0 rc6 kernel built successfully. Image at arch/x86/boot/bzImage (15MB).

Step 5: Why a Guest OS Was Needed

QEMU emulates the CXL hardware at the machine level. This means it creates a virtual CXL host bridge, root port, and memory device at the hardware abstraction level. To actually see and interact with this emulated hardware, a full operating system is needed inside QEMU because:

Requirement	Why it requires a Guest OS
CXL kernel drivers	The CXL subsystem lives inside the Linux kernel. The guest OS kernel loads cxl_pci, cxl_acpi, and cxl_core drivers to discover and manage the emulated hardware. WSL2 kernel does not expose these drivers to QEMU.
ACPI table enumeration	QEMU generates ACPI tables (CEDT, SRAT, HMAT) that describe CXL topology. Only a full OS booting inside QEMU reads these tables during boot to discover the CXL hierarchy.
CXL tools (cxl, ndctl)	Commands like cxl list, ndctl list, and daxctl interact with the CXL kernel subsystem via sysfs and ioctls. These only work inside the VM where the CXL hardware is visible.
PCIe enumeration	The CXL device appears on the PCIe bus of the virtual machine. Only the guest OS performs PCIe bus enumeration and assigns the device to the correct driver.

In simple terms: QEMU creates the fake CXL hardware, but without a full OS inside that VM, there is nothing to discover, initialize, or talk to that hardware. The guest OS (Ubuntu 24.04) serves as the runtime environment that brings the emulated CXL stack to life.

The guest OS setup commands:

```
wget https://cloud-images.ubuntu.com/noble/current/noble-server-cloudimg-amd64.img
qemu-img resize ~/cxl/noble-server-cloudimg-amd64.img +10G
cloud-localds ~/cxl/seed.img ~/cxl/cloud-init.img
```

Step 6: Create CXL Memory Backing Files

Two files were created to back the emulated CXL device memory and label storage:

```
truncate -s 512M ~/cxl/cxl-mem1.img    # CXL device memory (512MB)
truncate -s 4K   ~/cxl/cxl-label1.img  # CXL label storage area (4KB)
```

Step 7: Launch QEMU with CXL Emulation

The complete QEMU command to launch the VM with CXL Type 3 memory device:

```
~/cxl/qemu/build/qemu-system-x86_64 \
  -kernel ~/cxl/linux/arch/x86/boot/bzImage \
  -append "root=/dev/vda1 rw console=ttyS0 nokaslr" \
  -nographic -serial mon:stdio \
  -m 2G,slots=8,maxmem=8G -smp 4 \
  -accel tcg,thread=multi \
  -M q35,cxl=on,cxl-fmw.0.targets.0=cxl.1,cxl-fmw.0.size=4G \
  -object memory-backend-file,id=cxl-mem1,share=on,mem-path=/home/madhur/cxl/cxl-
mem1.img,size=512M \
  -object memory-backend-file,id=cxl-label1,share=on,mem-
path=/home/madhur/cxl/cxl-label1.img,size=4K \
  -device pxb-cxl,bus_nr=12,bus=pcie.0,id=cxl.1 \
  -device cxl-rp,port=0,bus=cxl.1,id=root_port13,chassis=0,slot=2 \
  -device cxl-type3,bus=root_port13,volatile-memdev=cxl-mem1,lsa=cxl-
label1,id=cxl-vmem0 \
  -drive file=~/cxl/noble-server-cloudimg-amd64.img,if=none,id=hd0,format=qcow2 \
  -device virtio-blk-pci,drive=hd0,bus=pcie.0 \
  -drive file=~/cxl/seed.img,if=none,id=seed0,format=raw \
  -device virtio-blk-pci,drive=seed0,bus=pcie.0
```

4. Results

4.1 CXL Device Discovery

```
$ sudo cxl list
[
  {
    "memdev": "mem0",
    "ram_size": 536870912,
    "serial": 0,
    "host": "0000:0d:00.0"
  }
]
```

4.2 Full CXL Topology

```
root0 (CXL Host Bridge  ACPI.CXL)
    └── port1 (CXL Root Port  depth:1)
            └── endpoint2 (CXL Endpoint  depth:2)
                    └── mem0 (CXL Type 3 Device  512MB)
decoder0.0: 4GB window, volatile+pmem+accel capable
```

4.3 PCIe Bus Visibility

```
$ lspci | grep -i cxl
0d:00.0 CXL: Intel Corporation Device 0d93 (rev 01)
```

4.4 Kernel CXL Messages

```
$ sudo dmesg | grep -i cxl
[1.617320] acpi ACPI0016:00: _OSC: OS supports [CXL11PortRegAccess
CXL20PortDevRegAccess]
[1.623797] acpi ACPI0016:00: _OSC: OS now controls [CXLMemErrorReporting]
[1.784924] pci0000:0c: host supports CXL
[2.357027] cxl_pci 0000:0d:00.0: CXL MCE unsupported
```

4.5 Summary of Results

Feature	Status	Details
CXL device discovery	Yes	mem0 visible with 512MB via cxl list
CXL topology enumeration	Yes	root0 port1 endpoint2 mem0
PCIe bus visibility	Yes	0d:00.0 CXL Intel Device 0d93
Kernel CXL driver	Yes	cxl_pci, cxl_acpi loaded successfully
ACPI CXL negotiation	Yes	OS controls CXL memory error reporting
CXL sysfs entries	Yes	All expected entries present
HDM Decoder visibility	Yes	decoder0.0 with 4GB window
cxl create-region	No	HDM decoder commit fails (TCG limitation)
DAX device usage	No	Requires successful region creation
CXL memory as RAM	No	Requires region and daxctl online-memory

5. Known Limitations and Root Cause

5.1 What is KVM and Why It Matters for CXL

KVM stands for Kernel-based Virtual Machine. It is a Linux kernel feature that allows the CPU to directly execute virtual machine instructions using hardware virtualization extensions built into modern processors (Intel VT-x or AMD-V). Without KVM, QEMU falls back to TCG (Tiny Code Generator), which is a pure software emulation mode where every single CPU instruction of the guest is translated and executed in software.

5.2 Why the Specific Error Occurred

The error encountered was:

```
cxl region: create_region: region0: failed to commit decode: No such device or address
```

The error happens because software emulation (TCG) doesn't have the exact timing of real hardware. Because of this slight delay, the system times out waiting for a confirmation when trying to create the memory region, causing it to fail. Using hardware virtualization (KVM) instead fixes this entirely.